

# Сравнение современных алгоритмов оптимизации

Колпащиков Максим  
Евгеньевич

студент 2 курса направления  
ИИНД ф-та матмех СПбГУ

Дмитрий Андреевич  
Михайлов

асс. кафедры информатики СПбГУ  
м. н. с. ЛПИИ СПб ФИЦ РАН



# Цель и задачи

**Цель:** Сравнить эффективность и особенности применения современных алгоритмов оптимизации AdaHessian, AdaBelief, AdaBound, AdamW, Lion при обучении нейронных сетей для задач регрессии, классификации и классификации изображений.

**Задачи:**

1. Кратко рассмотреть теоретические основы и ключевые особенности этих алгоритмов.
2. Реализовать программные модули для обучения моделей с использованием выбранных оптимизаторов и настройки их гиперпараметров.
3. Провести экспериментальное обучение моделей на различных задачах, подбирая оптимальные гиперпараметры для каждого оптимизатора.
4. Сравнить результаты по метрикам, качеству модели, стабильности обучения и обоснованности подбора гиперпараметров, визуализировав основные выводы.

# AdamW

Год выхода: 2017

Гиперпараметры:

- **lr** — скорость обучения (значение по умолчанию:  $1e-3$ );
- **betas** — коэффициенты для расчета экспоненциальных скользящих средних градиента и его квадрата (значения по умолчанию:  $(0.9, 0.999)$ );
- **epsilon** — параметр для численной стабильности в знаменателе (значение по умолчанию:  $1e-8$ );
- **weight\_decay** — коэффициент уменьшения весов (L2-регуляризация) (значение по умолчанию:  $1e-2$ );
- **amsgrad** — использование модификации AMSGrad (значение по умолчанию: False).

Идея:

Разделяет процесс уменьшения весов (weight decay) и обновление параметров градиентным спуском. В отличие от Adam, где L2-регуляризация добавляется к градиенту, в AdamW уменьшение весов применяется независимо. Это улучшает регуляризацию, особенно для адаптивных методов, и обеспечивает лучшую обобщающую способность и точность.

# AdaBelief

Год выхода: 2020

Гиперпараметры:

- **lr** — скорость обучения (значение по умолчанию:  $1e-3$ );
- **betas** — коэффициенты для расчета экспоненциальных скользящих средних градиента и его квадрата (значения по умолчанию: (0.9, 0.999));
- **epsilon** — параметр для численной стабильности в знаменателе (значение по умолчанию:  $1e-8$ );
- **weight\_decay** — коэффициент уменьшения весов (L2-регуляризация) (значение по умолчанию: 0);
- **amsgrad** — использование модификации AMSGrad (значение по умолчанию: False);
- **weight\_decouple** — использовать ли уменьшение весов как в AdamW (значение по умолчанию: False);
- **fixed\_decay** — используется, если параметр `weight_decay` установлен. Если True, веса уменьшаются (значение по умолчанию: False);
- **rectify** — выполняет обновление, аналогично RAdam (значение по умолчанию: False).

Идея:

Адаптирует размер шага в соответствии с "доверием" к текущему направлению градиента. Если наблюдаемый градиент сильно отличается от предсказанного (экспоненциального скользящего среднего), шаг делается маленьким; если близок – большим. Авторы утверждают о быстрой сходимости и высокой точности.



# AdaHessian

Год выхода: 2020

Гиперпараметры:

- **lr** — скорость обучения (значение по умолчанию: 0.15);
- **betas** — коэффициенты для расчета экспоненциальных скользящих средних градиента и его квадрата (значения по умолчанию: (0.9, 0.999));
- **epsilon** — параметр для численной стабильности в знаменателе (значение по умолчанию:  $1e-4$ );
- **weight\_decay** — коэффициент уменьшения весов (L2-регуляризация) (значение по умолчанию: 0);
- **hessian\_power** — степень Гесса (значение по умолчанию: 1);
- **spatial\_average\_block\_size** — Размер блока пространственного усреднения для одномерных тензоров (значения по умолчанию: (-1, -1, -1, -1)).

Идея:

Адаптивный оптимизатор второго порядка, учитывающий кривизну функции потерь. Вместо полной матрицы Гессе (вычислительно дорого), оценивает её диагональ с помощью метода Хатчинсона, что снижает затраты.

# AdaBound

Год выхода: 2019

Гиперпараметры:

- **lr** — скорость обучения (значение по умолчанию:  $1e-3$ );
- **betas** — коэффициенты для расчета экспоненциальных скользящих средних градиента и его квадрата (значения по умолчанию: (0.9, 0.999));
- **final\_lr** — конечная скорость обучения (SGD) (значение по умолчанию:  $1e-1$ );
- **gamma** — скорость сходимости для граничной функции (значение по умолчанию:  $1e-3$ );
- **epsilon** — параметр для численной стабильности в знаменателе (значение по умолчанию:  $1e-8$ );
- **weight\_decay** — коэффициент уменьшения весов (L2-регуляризация) (значение по умолчанию: 0);
- **amsgrad** — использование модификации AMSGrad (значение по умолчанию: False).

Идея:

AdaBound адаптирует скорость обучения с помощью динамических границ, переходя от адаптивных методов к SGD по мере обучения, что обеспечивает быструю сходимость и хорошую обобщающую способность.

# Lion

Год выхода: 2023

Гиперпараметры:

- **lr** — скорость обучения (значение по умолчанию:  $1e-4$ );
- **betas** — коэффициенты для расчета экспоненциальных скользящих средних градиента и его квадрата (значения по умолчанию: (0.9, 0.99));
- **weight\_decay** — коэффициент уменьшения весов (L2-регуляризация) (значение по умолчанию: 0);
- **use\_triton** - использовать Triton (значение по умолчанию: False);
- **decoupled\_weight\_decay** - использовать ли независимое уменьшение весов (значение по умолчанию: False).

Идея:

Использует знак градиента и инерцию для обновления весов. В отличие от Adam, применяет одинаковое значение обновления для каждого параметра, определяемое функцией знака. Это уменьшает потребление памяти (отслеживает только импульс) и может повысить скорость.

# Использованные технологии и реализация

Использованные библиотеки:

- **PyTorch**: Основной фреймворк для глубокого обучения
- **RayTune**: Для распределенного подбора гиперпараметров
- **TensorBoard**: Мониторинг процесса обучения и визуализация метрик в реальном времени.
- **Pandas**: Обработка и анализа данных
- **Seaborn**: Построение графиков и визуализация сравнительных результатов.

Программная реализация включала модули для:

- **Конфигурации** (config.py): Глобальные переменные, пути, seed для воспроизводимости.
- **Подготовки данных** (data\_setup.py): Загрузка, нормализация, создание Dataset и DataLoader.
- **Вспомогательных утилит** (utils.py): Создание логов TensorBoard, функции визуализации (графики потерь, метрик).
- **Построения моделей** (model\_builder.py): Реализация базовых моделей SimpleRegressionModel и SimpleClassificationModel.
- **Цикла обучения** (engine.py): Функции для шагов обучения и валидации, вычисления потерь, обратного распространения ошибки.



# Условия экспериментов и исследовательские вопросы

Вычисления проводились на платформе Kaggle:

- ЦП: 4-ядерный Intel Xeon CPU @ 2.00GHz.
  - ГПУ: 2 x NVIDIA Tesla T4 (позволяло распараллеливать подбор гиперпараметров с RayTune).
  - ОЗУ: 32GB.
- Версии библиотек: PyTorch 2.1.2+, TorchVision 0.16.2+, Ray 2.9.3+, Pandas 2.0.3+, Seaborn 0.13.1+.

**RQ1:** Какой оптимизатор (AdaHessian, Lion, AdaBelief, AdamW, AdaBound) показывает лучшую производительность на задачах регрессии, табличной классификации и классификации изображений с подобранными и стандартными гиперпараметрами?

**RQ2:** Насколько существенна разница во времени обучения между оптимизаторами при достижении сопоставимого качества?

**RQ3:** Критичен ли подбор гиперпараметров для высокой производительности всех оптимизаторов, или некоторые показывают хороший результат с параметрами по умолчанию?

# Функции потерь и метрики

## Для регрессии:

- MAPE (Средняя абсолютная процентная ошибка).
- $R^2$  Score (Коэффициент детерминации).

## Для классификации (включая изображения):

- Accuracy (Точность).
- Precision (macro-averaged).
- Recall (macro-averaged).
- AUC ROC (One-vs-Rest).

**Общие:** Время обучения.

# Задача регрессии

**Набор данных:** California Housing Prices (20640 наблюдений, 8 числовых признаков, целевая переменная – MedianHouseValue). Делилось на train/valid/test в соотношении 0.68/0.17/0.15. Данные нормализованы с StandardScaler.

**Архитектура модели:** SimpleRegressionModel – небольшая полносвязная сеть из трех слоев с ReLU активациями после первых двух и линейным выходным слоем (всего 4,801 параметр).

**Настройка гиперпараметров:** Обучение 31 эпоху, batch size 128. RayTune использовался для подбора lr, weight\_decay и специфичных для оптимизатора параметров (например, hessian\_power для AdaHessian, final\_lr и gamma для AdaBound). Диапазоны поиска: lr от  $1e-7$  до 1.0, weight\_decay от 0 до  $1e-1$ .

# Результат подбора гиперпараметров

| Optimizer  | Config                                                                      | MAPE   | Time (sec) |
|------------|-----------------------------------------------------------------------------|--------|------------|
| AdaHessian | {'lr': 0.1, 'wd': 0, 'hp': 1.0}                                             | 0.2069 | 46.0535    |
| Lion       | {'lr': 0.1, 'wd': 1e-4, 'dwd': True, 'ut': False}                           | 0.2179 | 31.5534    |
| AdaBelief  | {'lr': 0.1, 'wd': 0.01, 'ag': True, 'wdec': True, 'r': False, 'pcl': False} | 0.2476 | 34.5164    |
| AdamW      | {'lr': 0.1, 'wd': 0.01, 'ag': False}                                        | 0.2475 | 31.0266    |
| AdaBound   | {'lr': 1e-5, 'flr': 1e-7, 'g': 0.01, 'wd': 1e-3, 'ab': False}               | 0.2078 | 35.023     |

Таблица 1: Лучшие конфигурации оптимизаторов по метрике MAPE для задачи регрессии

| Optimizer  | Config                                                                       | R2 Score | Time (sec) |
|------------|------------------------------------------------------------------------------|----------|------------|
| AdaHessian | {'lr': 0.01, 'wd': 1e-3, 'hp': 0.75}                                         | 0.7561   | 46.4112    |
| Lion       | {'lr': 0.1, 'wd': 1e-4, 'dwd': True, 'ut': False}                            | 0.7479   | 31.5534    |
| AdaBelief  | {'lr': 0.1, 'wd': 1e-3, 'ag': False, 'wdec': True, 'r': False, 'pcl': False} | 0.7062   | 33.9567    |
| AdamW      | {'lr': 0.1, 'wd': 1e-3, 'ag': False}                                         | 0.7047   | 30.9785    |
| AdaBound   | {'lr': 1e-4, 'flr': 1e-7, 'g': 0.01, 'wd': 0, 'ab': False}                   | 0.7299   | 33.7272    |

Таблица 2: Лучшие конфигурации оптимизаторов по метрике R2 Score для задачи регрессии



# Результаты обучения на тестовой выборке

| Optimizer            | MAPE   | R2 score | Time (sec) | Config                                                                                                                                 |
|----------------------|--------|----------|------------|----------------------------------------------------------------------------------------------------------------------------------------|
| AdaHessian (default) | 0.2364 | 0.7469   | 47.511     | {'lr': 0.15, 'betas': (0.9,0.999), 'eps': 1e-4, 'wd': 0, 'hp': 0.5}                                                                    |
| AdaHessian (tuned)   | 0.2315 | 0.753    | 48.781     | {'lr': 0.01, 'wd': 0.001, 'hp': 0.75}                                                                                                  |
| Lion (default)       | 0.2376 | 0.7436   | 37.647     | {'lr': 1e-4, 'betas': (0.9,0.99), 'wd': 0.0, 'ut': False, 'dwd': False}                                                                |
| Lion (tuned)         | 0.2396 | 0.7317   | 37.631     | {'lr': 0.1, 'wd': 1e-4, 'dwd': True, 'ut': False}                                                                                      |
| AdaBelief (default)  | 0.2455 | 0.6734   | 38.372     | {'lr': 1e-3, 'betas': (0.9,0.999), 'eps': 1e-16, 'wd': 0, 'ag': False, 'wdec': True, 'fd': False, 'r': True, 'dts': True, 'pcl': True} |
| AdaBelief (tuned)    | 0.2592 | 0.6683   | 38.535     | {'lr': 0.1, 'wd': 0.001, 'ag': False, 'wdec': True, 'r': False, 'pcl': False}                                                          |
| AdamW (default)      | 0.257  | 0.6733   | 36.254     | {'lr': 1e-3, 'betas': (0.9,0.999), 'eps': 1e-08, 'wd': 0.01, 'ag': False}                                                              |
| AdamW (tuned)        | 0.2823 | 0.6591   | 36.963     | {'lr': 0.1, 'wd': 0.001, 'ag': False}                                                                                                  |
| AdaBound (default)   | 0.2463 | 0.6953   | 38.596     | {'lr': 1e-3, 'betas': (0.9,0.999), 'flr': 0.1, 'g': 1e-3, 'eps': 1e-08, 'wd': 0, 'ab': False}                                          |
| AdaBound (tuned)     | 0.2097 | 0.6971   | 38.588     | {'lr': 1e-4, 'flr': 1e-7, 'g': 0.01, 'wd': 0, 'ab': False}                                                                             |

Таблица 3: Сравнение подобранных и стандартных конфигураций оптимизаторов на тестовой выборке для задачи регрессии

# Визуализация результатов

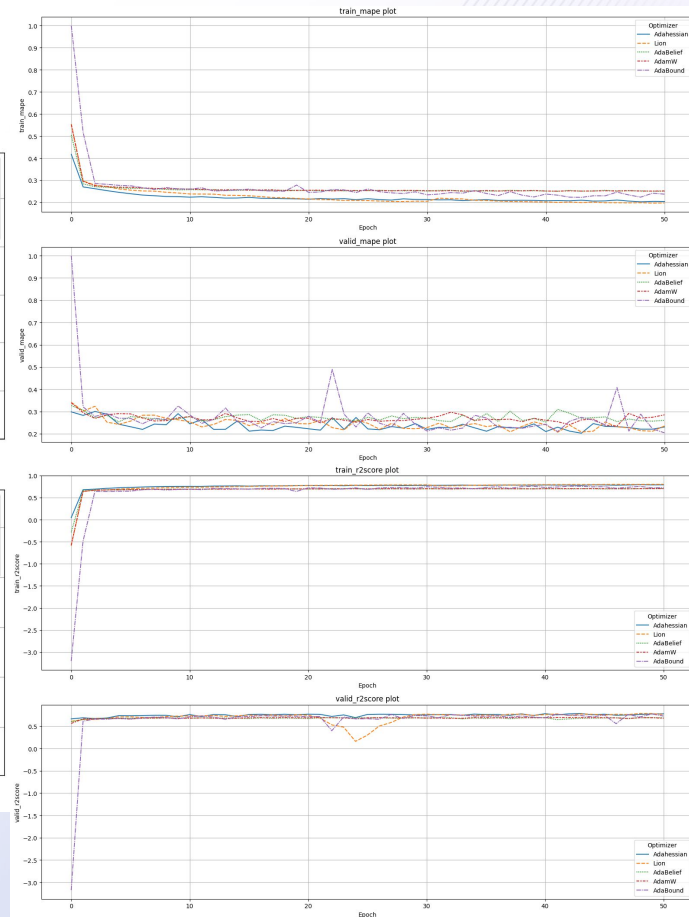
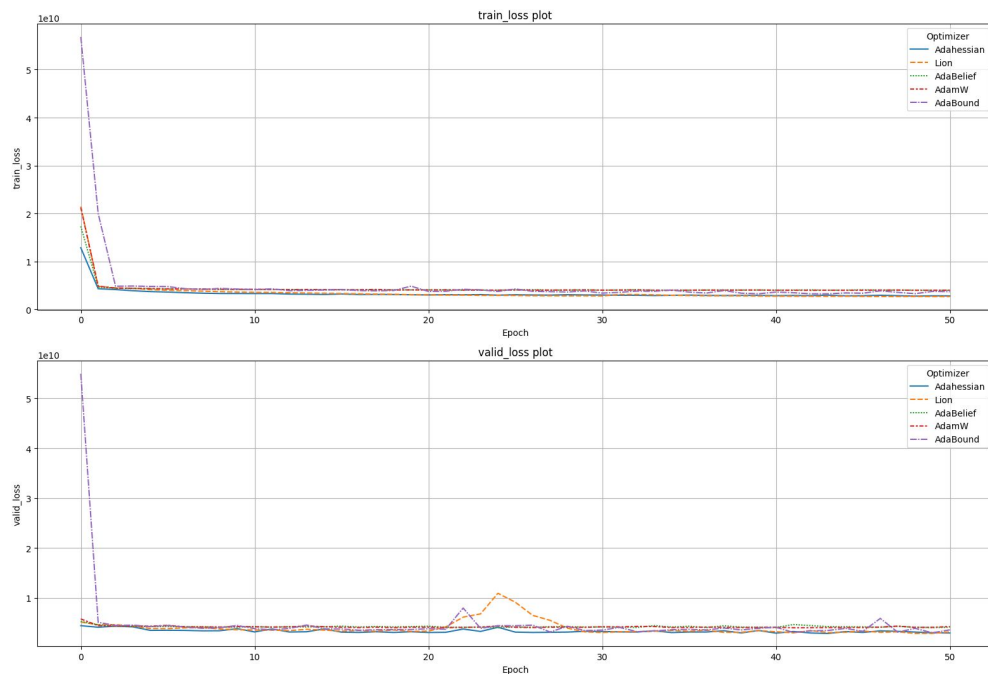


Рисунок 1: Динамика метрик обучения и валидации для различных оптимизаторов на задаче регрессии. По оси X -- эпохи.

# Задача классификации

**Набор данных:** Covertypes (типы лесного покрова), использовалась случайная выборка 10% (58101 наблюдение). Затем эта выборка разделилась на соотношение 0.72/0.18/0.1; 54 признака (10 количественных, 44 бинарных), 7 классов. Количественные признаки нормализованы.

**Архитектура модели:** SimpleClassificationModel – полносвязная сеть из двух слоев (всего 3,355 параметров) с ReLU функциями активациями, выходной слой на 7 нейронов.

**Настройка гиперпараметров:** Обучение 26 эпох, batch size 256. RayTune для подбора lr, weight\_decay и др. Диапазоны: lr от  $1e-4$  до 1.0, weight\_decay от 0 до  $1e-2$ .

# Результат подбора гиперпараметров

| Optimizer  | Config                                                                        | Accuracy | Time (seconds) |
|------------|-------------------------------------------------------------------------------|----------|----------------|
| AdaHessian | {'lr': 0.15, 'wd': 0, 'hp': 0.5}                                              | 0.7918   | 56.416         |
| Lion       | {'lr': 0.001, 'wd': 0.001, 'dwd': False, 'ut': False}                         | 0.7971   | 43.788         |
| AdaBelief  | {'lr': 0.01, 'wd': 0.001, 'ag': True, 'wdec': True, 'r': False, 'pcl': False} | 0.7876   | 46.664         |
| AdamW      | {'lr': 0.01, 'wd': 0.0001, 'ag': True}                                        | 0.7866   | 45.097         |
| AdaBound   | {'lr': 0.01, 'flr': 0.1, 'g': 0.0001, 'wd': 0.0001, 'ab': False}              | 0.7899   | 47.705         |

Таблица 4: Лучшие конфигурации оптимизаторов по метрике Ассигасу для задачи классификации



# Результаты обучения на тестовой выборке

| Optimizer            | Accuracy | Precision | Recall | Time (sec) | Config                                                                                                         |
|----------------------|----------|-----------|--------|------------|----------------------------------------------------------------------------------------------------------------|
| AdaHessian (default) | 0.799    | 0.7327    | 0.6725 | 86.5       | {'lr': 0.15, 'b': (0.9,0.999), 'eps': 1e-4, 'wd': 0, 'hp': 0.5}                                                |
| AdaHessian (tuned)   | 0.8014   | 0.7735    | 0.6357 | 84.65      | {'lr': 0.15, 'wd': 0, 'hp': 0.5}                                                                               |
| Lion (default)       | 0.768    | 0.6604    | 0.5591 | 73.32      | {'lr': 1e-4, 'b': (0.9,0.99), 'wd': 0, 'ut': F, 'dwd': F}                                                      |
| Lion (tuned)         | 0.8031   | 0.7376    | 0.6646 | 71.43      | {'lr': 0.001, 'wd': 0.001, 'dwd': F, 'ut': F}                                                                  |
| AdaBelief (default)  | 0.7754   | 0.7429    | 0.5922 | 75.78      | {'lr': 1e-3, 'b': (0.9,0.999), 'eps': 1e-16, 'wd': 0, 'ag': F, 'wdec': T, 'fd': F, 'r': T, 'dts': T, 'pcl': T} |
| AdaBelief (tuned)    | 0.8073   | 0.7411    | 0.6745 | 72.76      | {'lr': 0.01, 'wd': 0.001, 'ag': T, 'wdec': T, 'r': F, 'pcl': F}                                                |
| AdamW (default)      | 0.7806   | 0.7014    | 0.5841 | 73.59      | {'lr': 1e-3, 'b': (0.9,0.999), 'eps': 1e-8, 'wd': 0.01, 'ag': F}                                               |
| AdamW (tuned)        | 0.7981   | 0.7406    | 0.6815 | 71.48      | {'lr': 0.01, 'wd': 1e-4, 'ag': T}                                                                              |
| AdaBound (default)   | 0.7785   | 0.689     | 0.5911 | 75.21      | {'lr': 1e-3, 'b': (0.9,0.999), 'flr': 0.1, 'g': 1e-3, 'eps': 1e-8, 'wd': 0, 'ab': F}                           |
| AdaBound (tuned)     | 0.7945   | 0.7115    | 0.6476 | 74.97      | {'lr': 0.01, 'flr': 0.1, 'g': 1e-4, 'wd': 1e-4, 'ab': F}                                                       |

Таблица 5: Сравнение подобранных и стандартных конфигураций оптимизаторов на тестовой выборке для задачи классификации

# Визуализация результатов

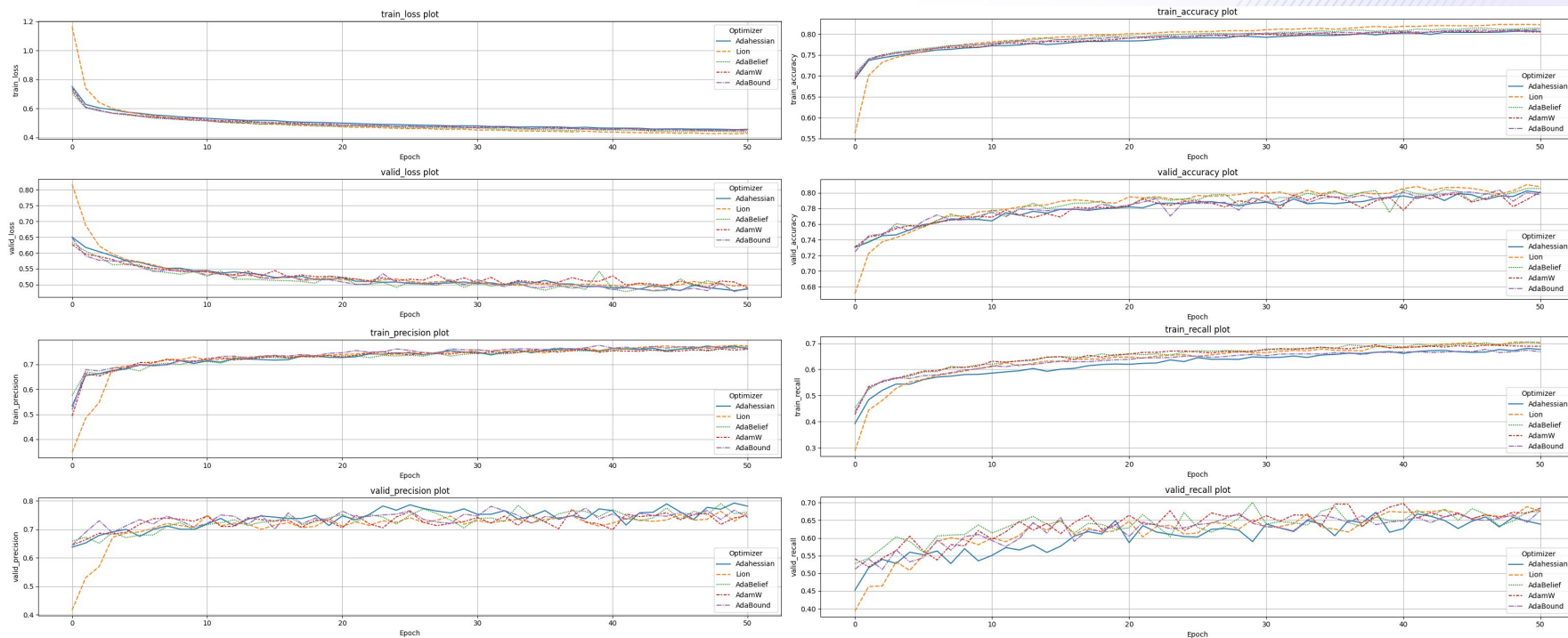


Рисунок 2: Динамика метрик обучения и валидации для различных оптимизаторов на задаче классификации. По оси X -- эпохи.

# Задача классификации изображений

**Набор данных:** CIFAR-10 (60000 цветных изображений 32x32, 10 классов; 50000 для обучения, 10000 для теста). Разбиение выборки было на 66/17/17. Изображения нормализованы.

**Архитектура модели:** ResNet-18, последний слой заменен для 10 классов CIFAR-10.

**Настройка гиперпараметров (начальная, 5 эпох):** Batch size 256.

RayTune для подбора `lr`, `weight_decay` и др. Диапазоны: `lr` от  $1e-5$  до 0,15, `weight_decay` от 0 до  $1e-1$ .

# Результат подбора гиперпараметров

| Optimizer  | Config                                                                        | Accuracy | Time (seconds) |
|------------|-------------------------------------------------------------------------------|----------|----------------|
| AdaHessian | {'lr': 0.15, 'wd': 0.0001, 'hp': 0.75}                                        | 0.6923   | 170.39         |
| Lion       | {'lr': 0.001, 'wd': 0.001, 'dwd': False, 'ut': False}                         | 0.7223   | 69.28          |
| AdaBelief  | {'lr': 0.01, 'wd': 0.001, 'ag': False, 'wdec': True, 'r': True, 'pcl': False} | 0.7117   | 72.12          |
| AdamW      | {'lr': 0.001, 'wd': 0, 'ag': False}                                           | 0.6924   | 68.42          |
| AdaBound   | {'lr': 0.001, 'flr': 0.1, 'g': 0.0001, 'wd': 0.0001, 'ab': False}             | 0.6988   | 73.98          |

Таблица 6: Лучшие конфигурации оптимизаторов по метрике Ассигасу для задачи классификации изображений

# Результаты обучения на тестовой выборке

| Optimizer            | Accuracy | Precision | Recall | Time   | Config                                                                                                          |
|----------------------|----------|-----------|--------|--------|-----------------------------------------------------------------------------------------------------------------|
| AdaHessian (default) | 0.7198   | 0.7271    | 0.7198 | 689.65 | {'lr': 0.15, 'b': (0.9, 0.999), 'eps': 1e-4, 'wd': 0, 'hp': 0.5}                                                |
| AdaHessian (tuned)   | 0.7283   | 0.733     | 0.7283 | 684.62 | {'lr': 0.15, 'wd': 1e-4, 'hp': 0.75}                                                                            |
| Lion (default)       | 0.7073   | 0.7078    | 0.7073 | 238.96 | {'lr': 1e-4, 'b': (0.9, 0.99), 'wd': 0, 'ut': F, 'dwd': F}                                                      |
| Lion (tuned)         | 0.7324   | 0.7353    | 0.7324 | 239.82 | {'lr': 0.001, 'wd': 0.001, 'dwd': F, 'ut': F}                                                                   |
| AdaBelief (default)  | 0.6896   | 0.6879    | 0.6896 | 245.83 | {'lr': 1e-3, 'b': (0.9, 0.999), 'eps': 1e-16, 'wd': 0, 'ag': F, 'wdec': T, 'fd': F, 'r': T, 'dts': T, 'pcl': T} |
| AdaBelief (tuned)    | 0.7531   | 0.756     | 0.7531 | 245.2  | {'lr': 0.01, 'wd': 0.001, 'ag': F, 'wdec': T, 'r': T, 'pcl': F}                                                 |
| AdamW (default)      | 0.7129   | 0.7266    | 0.7129 | 238.72 | {'lr': 1e-3, 'b': (0.9, 0.999), 'eps': 1e-8, 'wd': 0.01, 'ag': F}                                               |
| AdamW (tuned)        | 0.7236   | 0.7239    | 0.7236 | 238.73 | {'lr': 0.001, 'wd': 0, 'ag': F}                                                                                 |
| AdaBound (default)   | 0.7190   | 0.7205    | 0.7190 | 243.85 | {'lr': 1e-3, 'b': (0.9, 0.999), 'flr': 0.1, 'g': 1e-3, 'eps': 1e-8, 'wd': 0, 'ab': F}                           |
| AdaBound (tuned)     | 0.7377   | 0.7404    | 0.7377 | 246.82 | {'lr': 0.001, 'flr': 0.1, 'g': 1e-4, 'wd': 1e-4, 'ab': F}                                                       |

Таблица 7: Сравнение подобранных и стандартных конфигураций оптимизаторов на тестовой выборке для задачи классификации изображений



# Визуализация результатов

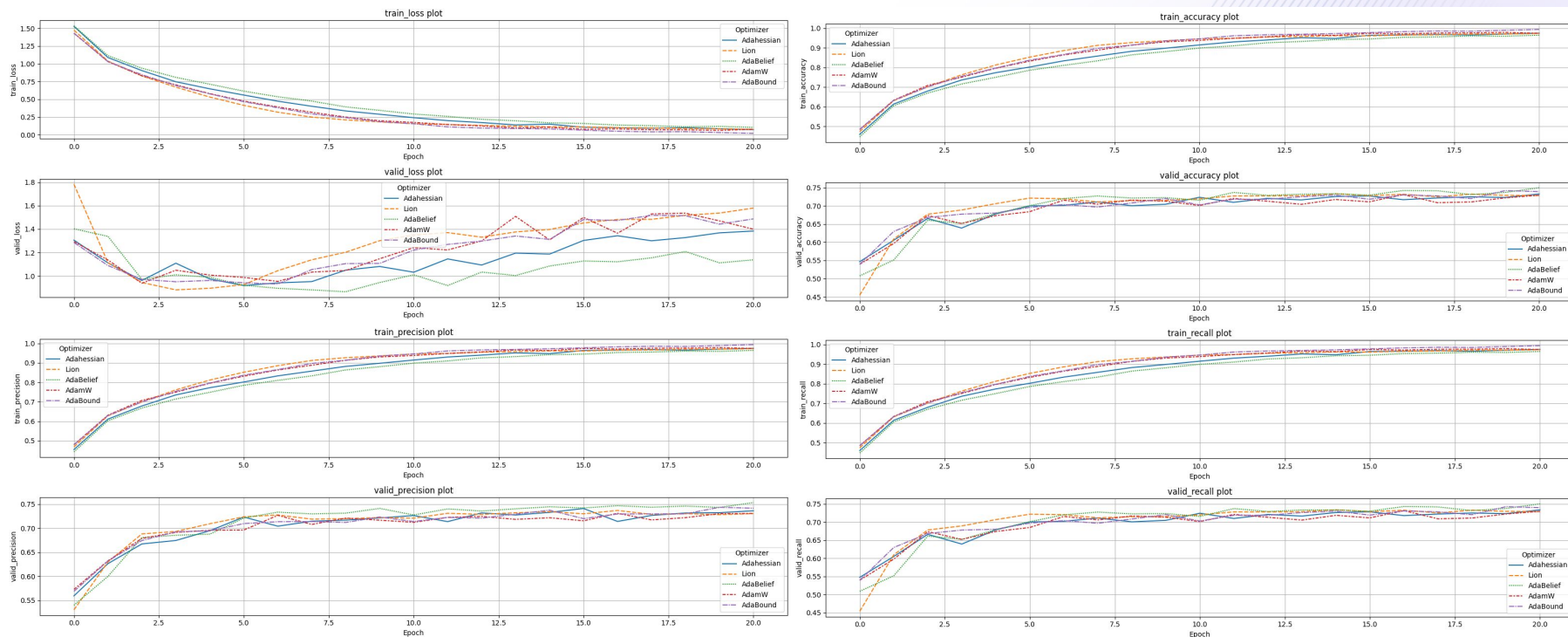


Рисунок 3: Динамика метрик обучения и валидации для различных оптимизаторов на задаче классификации изображений. По оси X -- эпохи.

# Ответы на исследовательские вопросы

**RQ1: Какой оптимизатор демонстрирует наилучшую производительность на различных задачах?**

В целом, **AdaBelief** и **Lion** часто показывали сильные результаты, особенно на задачах классификации. Единого доминирующего оптимизатора для всех задач нет.

**RQ2: Насколько существенна разница во времени обучения при достижении сопоставимого качества?**

Разница была существенной.

**RQ3: Является ли подбор гиперпараметров критически важным?**

Для сложных задач (классификация, классификация изображений с глубокими моделями) подбор гиперпараметров важен. На более простых задачах некоторые оптимизаторы могут хорошо работать и с параметрами по умолчанию.

# Заключение

Выбор оптимального алгоритма оптимизации зависит от задачи, архитектуры модели, данных и ресурсов.

- **AdaBelief** часто демонстрировал одни из лучших результатов по качеству на задачах классификации при конкурентоспособном времени.
- **Lion** был сильным конкурентом, часто обеспечивая лучшее или близкое к нему время обучения и высокое качество.
- **AdamW** обеспечивал высокую скорость, но по качеству на классификации с подобранными параметрами уступал AdaBelief и Lion.
- **AdaBound** хорошо показал себя на регрессии.
- **AdaHessian** требовал значительно больше времени, что ограничивает его применение, несмотря на конкурентоспособное качество после подбора только на регрессии.

Важность подбора гиперпараметров, особенно для задач классификации и классификации изображений, была подтверждена.